



دانشگاه صنعتی شریف
دانشکده مهندسی کامپیوتر

عنوان:

استفاده از حافظه داده و دستورات پرش

نام درس

آزمایشگاه معماری کامپیوتر

نیم سال ۱۴۰۴

استاد

دکتر سربازی آزاد

گروه ۲

امیرشایان سلیمانی نیا - ۴۰۳۱۰۶۱۰۸

محمد مهدی مرادی - ۴۰۳۱۰۶۶۸۱

فهرست مطالب

۱	مقدمه و هدف آزمایش	۲
۲	شرح آزمایش	۲
۳	ساختار مدار و طراحی زیرمدارها	۳
۱.۳	زیرمدار Unpacker	۳
۲.۳	زیرمدار Reg_File	۵
۳.۳	زیرمدار ALU	۶
۴.۳	زیرمدار DP	۷
۵.۳	زیرمدار CU	۸
۶.۳	مدار Main	۹
۴	تست‌های عملکرد سیستم و نتایج	۱۰
۱.۴	تست اول: محاسبه مجموع سری فیبوناچی	۱۰
۲.۴	تست دوم: جمع دو عدد ۶۴ بیتی	۱۳
۵	نتیجه‌گیری	۱۴

۱ مقدمه و هدف آزمایش

در آزمایش قبلی، به دلیل نبود حافظه داده، امکان ذخیره مقادیر میانی را نداشتیم. همچنین عدم وجود دستورات پرش باعث می‌شد نتوانیم در برنامه‌ها حلقه ایجاد کنیم.

بنابراین، هدف اصلی این آزمایش تکمیل مدار قبلی و ارتقای آن به یک کامپیوتر ساده است. در این ارتقا، قابلیت دسترسی به حافظه داده‌ها (برای خواندن و ذخیره مقادیر) و همچنین امکان استفاده از دستورات Jump شرطی و غیرشرطی به سیستم اضافه می‌شود.

۲ شرح آزمایش

سیستم طراحی شده در این آزمایش بر اساس معماری Harvard کار می‌کند؛ به این معنی که حافظه برنامه (ROM) از حافظه داده‌ها کاملاً مجزا است. برای بخش داده‌ها از یک RAM با ظرفیت ۳۲ بیت استفاده کرده‌ایم.

از لحاظ ساختار، این مدار شامل یک ALU هشت بیتی، چهار ثبات (R_0 تا R_3)، واحد کنترل، و یک ثبات وضعیت (SR) برای نگهداری فلگ‌ها C, Z, S, O است.

دستورالعمل‌هایی که این مدار قادر به اجرای آن‌هاست به سه گروه کلی تقسیم می‌شوند:

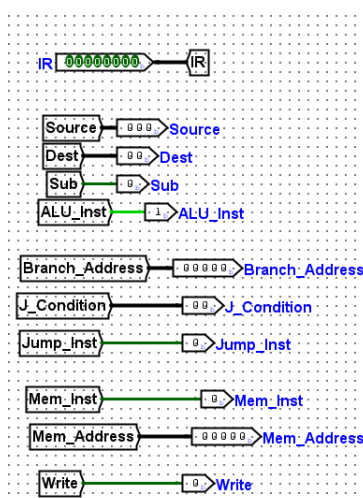
- **دستورات محاسباتی-انتقالی:** برای انجام عملیات حسابی یا جابجایی داده‌ها بین ثبات‌ها و ورودی‌های ALU.
- **دستورات دسترسی به حافظه داده:** برای بارگذاری محتوا از RAM به ثبات R_0 و یا ذخیره مقدار R_0 در RAM.
- **دستورات Jump (شرطی و غیرشرطی):** برای تغییر روند اجرای برنامه و پرش به آدرس‌های مختلف. در پرش‌های شرطی، مالتی‌پلکسر مدار کنترل با چک کردن فلگ‌ها ثبات وضعیت، تصمیم می‌گیرد که سیگنال PC Load را فعال کند یا خیر.

۳ ساختار مدار و طراحی زیرمدارها

۱.۳ زیرمدار Unpacker

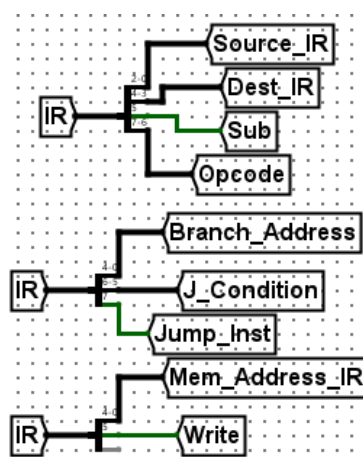
وظیفه اصلی زیرمدار Unpacker، دیکود کردن دستورات است. این ماژول، دستور ۸ بیتی قرار گرفته در IR را دریافت کرده و آن را به سیگنال‌های کنترلی و فیلدهای مورد نیاز (مثل آدرس ثبات‌ها یا آدرس حافظه) برای سایر بخش‌های پردازنده تجزیه می‌کند.

خروجی‌های آن شامل مقادیر تفکیک‌شده‌ای مانند Opcode، Source، Dest و سیگنال‌های تعیین‌کننده نوع دستور (ALU_Inst و Mem_Inst) می‌باشد.



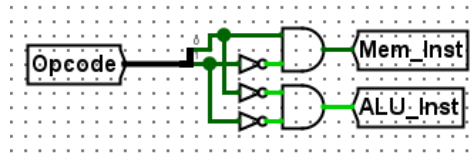
شکل ۱: ورودی‌ها و خروجی‌های زیرمدار Unpacker

برای استخراج فیلدهای مختلف از درون دستور، از Splitter استفاده می‌کنیم. با توجه به اینکه دستورات این پردازنده در قالب‌های مختلفی (محاسباتی، مموری و جامپ) تعریف شده‌اند، بیت‌های IR را به سه شکل متفاوت سیم‌کشی و تجزیه کرده‌ایم. با این کار مقادیر خامی نظیر Source_IR، Branch_Address و Mem_Address_IR به صورت همزمان استخراج می‌شوند.



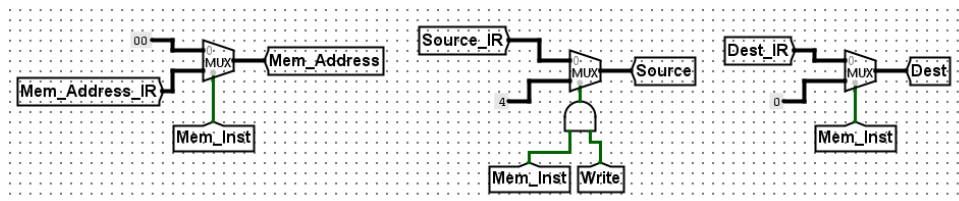
شکل ۲: تجزیه بیت‌های IR برای قالب‌های مختلف دستورات

در مرحله بعد، برای تشخیص دقیق نوع دستور، دو بیت پرارزش (Opcode) را بررسی می‌کنیم. اگر Opcode برابر 01 باشد، سیگنال Mem_Inst فعال می‌شود و اگر برابر 00 باشد، سیگنال ALU_Inst یک خواهد شد.



شکل ۳: تشخیص نوع دستور از روی Opcode

چالشی که در اینجا وجود دارد، تداخل فیلدها در قالب‌های مختلف است. مثلاً ممکن است ۵ بیت آخر دستور در یک عملیات محاسباتی به عنوان آدرس ثبات‌ها خوانده شود، در حالی که در واقع آدرس حافظه است. برای جلوگیری از این تداخل، از MUX استفاده کرده‌ایم تا فیلدها را فیلتر کنیم. به عنوان مثال، اگر سیگنال Mem_Inst فعال باشد، MUX مقدار ثابت 0 را به خروجی Dest می‌فرستد. همچنین در دستورات Store (زمانی که Mem_Inst و Write هر دو یک هستند)، مقدار Source روی 4 تنظیم می‌شود تا مسیر دیتای صحیحی در ALU شکل بگیرد. در صورتی که دستور از نوع حافظه نباشد، همان مقادیر استخراج شده از IR مستقیماً به خروجی می‌روند.

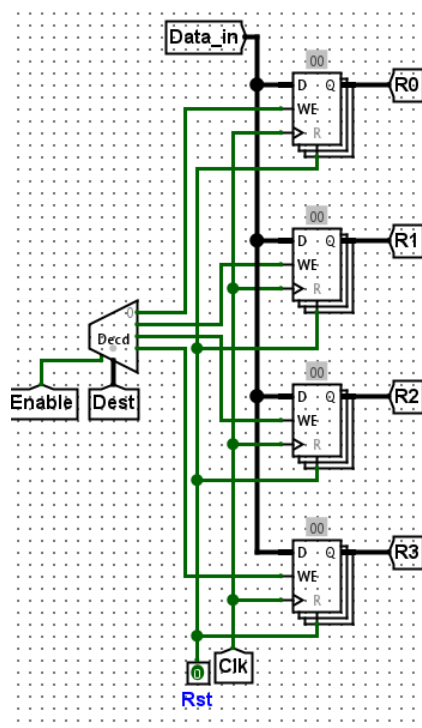


شکل ۴: استفاده از MUX برای مسیریابی و فیلتر کردن مقادیر بر اساس نوع دستور

۲.۳ زیرمدار Reg_File

زیرمدار Reg_File که وظیفه نگهداری ثبات‌های را بر عهده دارد، در آزمایش‌های قبلی نیز وجود داشت و ساختار آن حفظ شده است. تغییر مهمی که در این مرحله روی این زیرمدار اعمال شده، اضافه شدن سیگنال کنترلی Enable به ورودی فعال‌ساز مدار دیکودر است.

دلیل این تغییر، تنوع یافتن دستورات در این آزمایش است. در جریان اجرای برخی دستورات، مانند دستورات Jump، هیچ محاسباتی که نیاز به ذخیره در ثبات‌ها داشته باشد انجام نمی‌گیرد. برای جلوگیری از تغییر ناخواسته مقادیر ثبات‌ها در این حالت‌ها، سیگنال Enable غیرفعال می‌شود. با خاموش شدن دیکودر، هیچ‌کدام از پایه‌های WE در ثبات‌ها سیگنال دریافت نمی‌کنند و Data_in روی آن‌ها نوشته نخواهد شد.

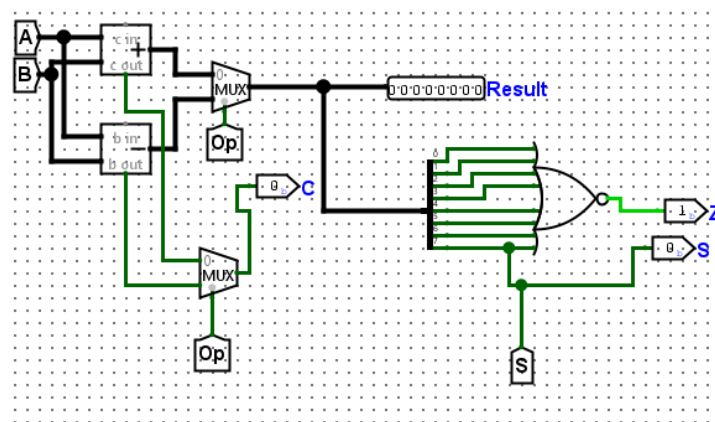


شکل ۵: ساختار زیرمدار Reg_File با اضافه شدن سیگنال Enable

۳.۳ زیرمدار ALU

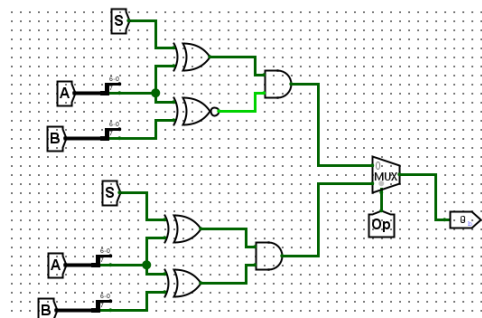
زیرمدار ALU در آزمایش‌های قبلی نیز وجود داشت، اما در آن زمان تنها وظیفه محاسبه و تولید خروجی جمع و تفریق را بر عهده داشت. در این آزمایش، ساختار آن ارتقا یافته تا سیگنال‌های وضعیت (فلگ‌ها C، Z، S و O) را نیز برای استفاده در دستورات شرطی تولید کند.

در مسیر اصلی داده، ورودی‌های A و B به یک جمع‌کننده و یک تفریق‌کننده متصل هستند و یک مالتی‌پلکسر با دریافت سیگنال Op، نتیجه نهایی را روی Result قرار می‌دهد. سیگنال‌های C یا Z نیز به کمک یک مالتی‌پلکسر دیگر از بین خروجی‌های جمع‌کننده یا تفریق‌کننده انتخاب می‌شود. برای فلگ Z، هشت بیت خروجی وارد یک گیت NOR هشت‌ورودی شده‌اند تا صفر بودن کل جواب را بررسی کنند و فلگ S نیز مستقیماً از باارزش‌ترین بیت خروجی گرفته شده است.



شکل ۶: مسیر اصلی داده در ALU و نحوه استخراج فلگ‌های C، Z، S

مدار تشخیص فلگ O (Overflow) به صورت مجزا با گیت‌های منطقی به این زیرمدار اضافه شده است. این بخش با بررسی بیت علامت عملوندها (بیت هفتم A و B) و مقایسه آن با علامت جواب نهایی، اورفلو را تشخیص می‌دهد. منطق مدار به این صورت است که اگر دو عدد هم‌علامت جمع شوند یا دو عدد با علامت‌های متفاوت از هم کم شوند و علامت جواب تغییر کند، اورفلو رخ داده است. در نهایت خروجی اورفلو مربوط به جمع یا تفریق، بسته به سیگنال Op انتخاب شده و به خروجی می‌رود.

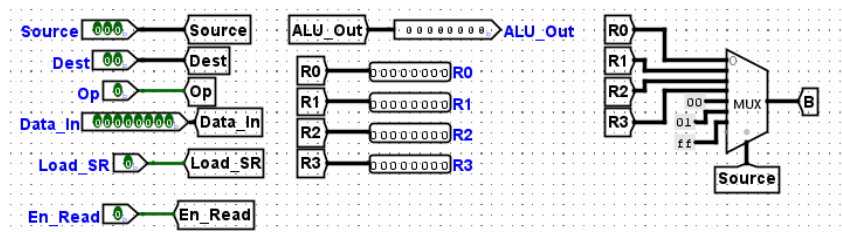


شکل ۷: مدار تشخیص اورفلو

۴.۳ زیرمدار DP

زیرمدار DP، عامل اصلی ارتباط بین ثبات‌ها و واحد محاسبه است. این بخش نسبت به آزمایش‌های قبلی شامل چند تغییر شده است.

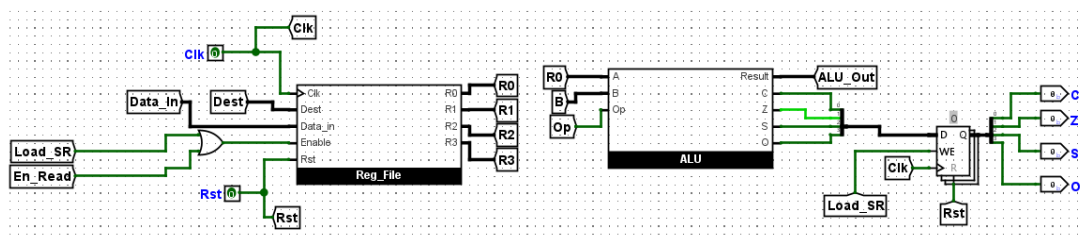
تغییر اول در نحوه دریافت ورودی‌هاست. در آزمایش‌های گذشته، یک دستور خام ۶ بیتی مستقیماً وارد این مدار می‌شد و استخراج فیلدها در همین بخش صورت می‌گرفت؛ اما اکنون اطلاعات مورد نیاز مانند Source، Dest و Op به صورت تفکیک‌شده دریافت می‌شوند. این سیگنال‌ها در واقع همان اطلاعاتی هستند که توسط زیرمدار Unpacker از روی دستور اصلی تجزیه و ارسال شده‌اند.



شکل ۸: ورودی‌های مستقل تفکیک‌شده و مسیر انتخاب عملوند دوم ALU

تغییر دوم مربوط به کنترل دقیق‌تر عملیات نوشتن در ثبات‌هاست. همان‌طور که پیش‌تر بررسی شد، به Reg_File یک پایه Enable اضافه شده است. در اینجا این پایه توسط یک گیت OR کنترل می‌شود که ورودی‌های آن سیگنال‌های Load_SR و En_Read هستند. این طراحی باعث می‌شود که داده‌های درون ثبات‌ها فقط در زمان اجرای دستورات محاسباتی یا خواندن از حافظه تغییر کنند و در سایر مواقع دست‌نخورده باقی بمانند.

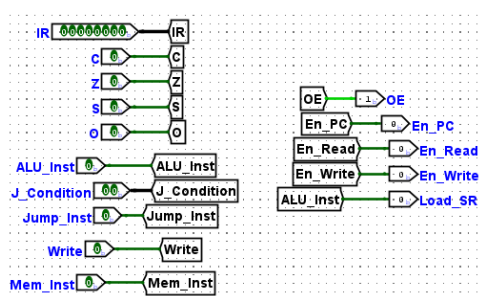
تغییر سوم، نحوه مدیریت فلگ‌های وضعیت است. خروجی فلگ‌ها از ALU دیگر به صورت مستقیم به خروجی مدار متصل نیست. در عوض، فلگ‌ها C, Z, S, O ابتدا وارد یک رجیستر ۴ بیتی می‌شوند. این رجیستر با فعال شدن سیگنال Load_SR، مقادیر فلگ‌ها را در خود قفل و ذخیره می‌کند. این کار به پردازنده اجازه می‌دهد تا نتایج وضعیت آخرین عملیات محاسباتی را برای استفاده در دستورات بعدی (مثل پرش‌های شرطی) نگه دارد.



شکل ۹: ارتباط Reg_File و ALU و نحوه ذخیره‌سازی فلگ‌ها در رجیستر وضعیت

۵.۳ زیرمدار CU

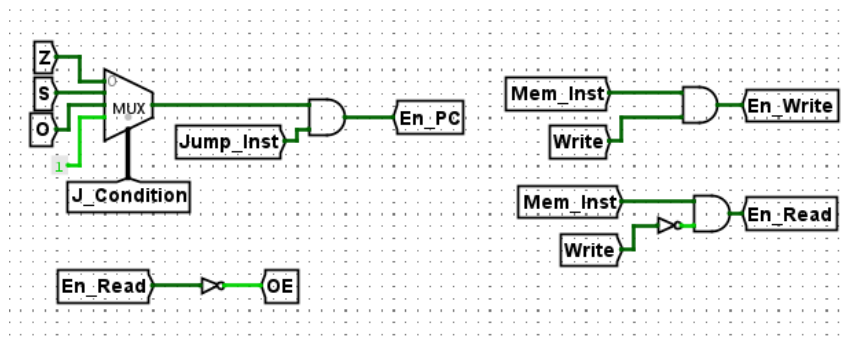
زیرمدار CU، وظیفه صدور سیگنال‌های کنترلی نهایی را بر اساس نوع دستور و وضعیت جاری سیستم بر عهده دارد. ورودی‌های این واحد شامل سیگنال‌های دیکود شده توسط Unpacker (مانند Mem_Inst، ALU_Inst، Write و Jump_Inst) به همراه فلگ‌ها وضعیت (Z, S, O, C) می‌باشند. خروجی‌های آن نیز سیگنال‌هایی نظیر En_Read، En_Write، Load_SR، En_PC، OE و En_PC هستند.



شکل ۱۰: نمای کلی ورودی‌ها و خروجی‌های واحد کنترل (CU)

منطق تولید سیگنال‌های مربوط به حافظه (En_Read و En_Write) بر پایه گیت‌های AND طراحی شده است. تنها در صورتی که سیگنال Mem_Inst فعال باشد، وضعیت بیت Write بررسی می‌شود. اگر Write یک باشد، سیگنال En_Write (مجوز نوشتن در RAM) فعال می‌شود و اگر صفر باشد، En_Read (مجوز خواندن از RAM) فعال می‌شود. سیگنال OE (Output Enable) که کنترل بافر سه‌حالته خروجی ALU را در دست دارد، دقیقاً معکوس En_Read در نظر گرفته شده است تا در زمان خواندن از حافظه، خروجی ALU روی باس داده تداخلی ایجاد نکند.

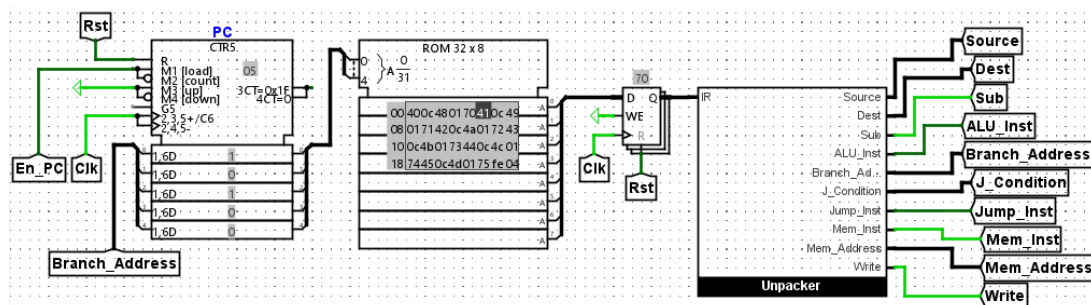
یکی از مهم‌ترین وظایف CU، مدیریت دستورات Jump است. برای این منظور، از یک مالتی‌پلکسر استفاده شده که ورودی‌های آن به ترتیب فلگ‌ها Z, S, O و عدد ثابت 1 (برای پرش غیرشرطی) هستند. سیگنال J_Condition به عنوان سلکتور این مالتی‌پلکسر عمل می‌کند. خروجی این مالتی‌پلکسر که نشان‌دهنده برقراری یا عدم برقراری شرط پرش است، با سیگنال Jump_Inst AND می‌شود. در صورتی که هم دستور از نوع پرش باشد و هم شرط آن برقرار باشد، سیگنال En_PC فعال شده و آدرس جدید در PC قرار می‌گیرد.



شکل ۱۱: منطق کنترلی برای تولید سیگنال‌های حافظه و پرش در CU

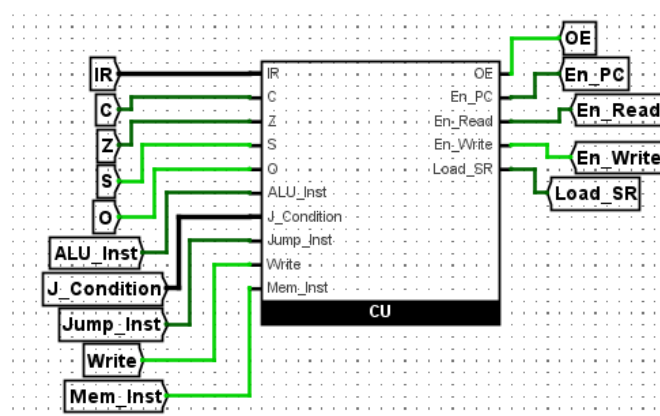
۶.۳ مدار Main

در مدار اصلی، تمام زیرمدارهایی که طراحی کردیم در کنار هم قرار می‌گیرند تا ساختار نهایی پردازنده شکل بگیرد. در بخش ابتدایی مدار، PC آدرس فعلی را به حافظه ROM می‌دهد تا دستور لود شود. این دستور در ثبات IR نگهداری شده و بلافاصله به ورودی Unpacker می‌رود تا به سیگنال‌ها و فیلدهای مورد نیاز تجزیه شود. در صورتی که دستور اجرا شده از نوع پرش باشد، Branch_Address از Unpacker مستقیماً به ورودی داده‌ی PC برمی‌گردد تا در پالس بعدی، اجرای برنامه از آن آدرس ادامه پیدا کند.



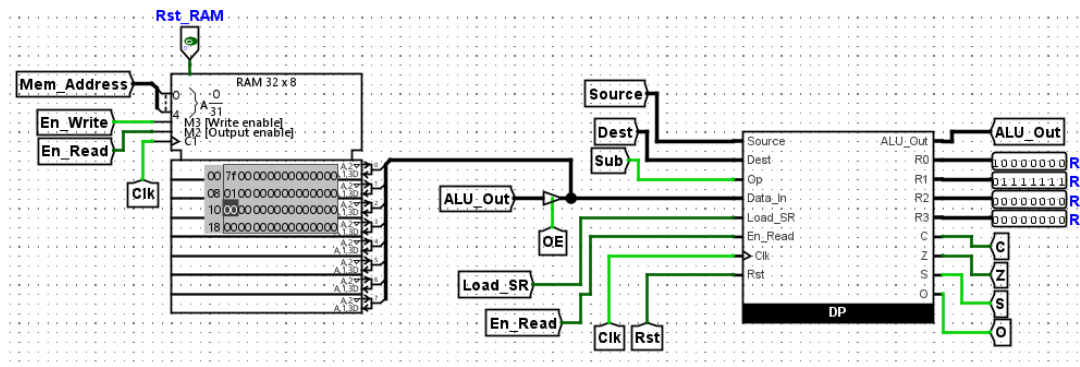
شکل ۱۲: بخش واکشی دستورات، حافظه ROM و زیرمدار Unpacker

واحد کنترل ارتباط بین بخش‌ها را مدیریت می‌کند. این واحد سیگنال‌های دیکود شده را از Unpacker و وضعیت فلگ‌ها (C, Z, S, O) را از DP دریافت کرده و سیگنال‌های کنترلی نهایی کل سیستم را صادر می‌کند.



شکل ۱۳: قرارگیری واحد کنترل و مدیریت سیگنال‌های سیستم

در بخش بعدی، تعامل بین RAM و زیرمدار DP برقرار شده است. یکی از مهم‌ترین نکات در این قسمت، مدیریت باس داده است. خروجی ALU و پورت داده‌ی RAM هر دو به یک باس مشترک متصل هستند که اطلاعات ورودی ثابت‌ها را مشخص می‌کند. برای جلوگیری از تداخل سیگنال‌ها روی این باس، خروجی ALU_Out از یک بافر سه‌حالت عبور می‌کند. زمانی که پردازنده دستور خواندن از RAM را اجرا می‌کند، سیگنال OE این بافر را در حالت قطع قرار می‌دهد و در عملیات محاسباتی، بافر وصل می‌شود تا نتیجه وارد DP گردد.



شکل ۱۴: ارتباط حافظه داده با DP و کنترل باس مشترک توسط بافر سه‌حالت

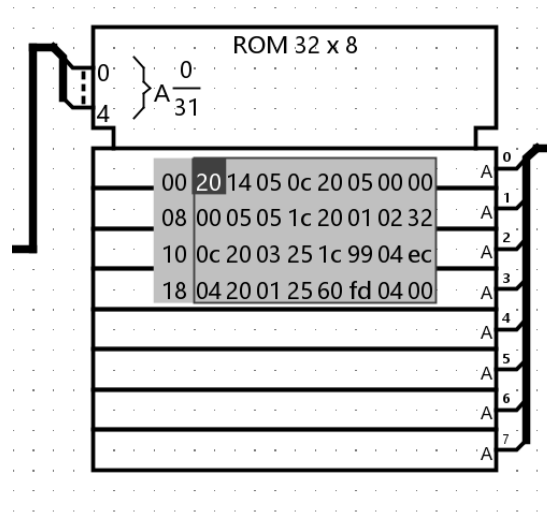
۴ تست‌های عملکرد سیستم و نتایج

۱.۴ تست اول: محاسبه مجموع سری فیبوناچی

الف) مجموع ده جمله اول سری فیبوناچی را محاسبه کرده و در آدرس صفر حافظه داده‌ها ذخیره کند (با استفاده از حلقه).

شکل ۱۵: خواسته بخش اول تست مدار

هدف این برنامه محاسبه مجموع ۱۰ جمله اول سری فیبوناچی با استفاده از ساختار حلقه و ذخیره آن در آدرس صفر حافظه داده است. مجموع n جمله اول سری فیبوناچی برابر است با جمله $(n + 2)$ ام منهای یک. بنابراین، تا جمله دوازدهم فیبوناچی پیش رفته و در نهایت یک واحد از آن کم می‌کنیم. کدهای هگزادسیمال مربوط به این برنامه درون حافظه ROM بارگذاری شده‌اند. این کدها را می‌توان در چهار فاز اصلی دسته‌بندی و تحلیل کرد:



شکل ۱۶: کدهای هگزادسیمال برنامه فیووناچی بارگذاری شده در ROM

۱. مقداردهی اولیه (آدرس‌های 0x00 تا 0x0B)

در این بخش، مقادیر اولیه سری فیووناچی و شمارنده حلقه تنظیم می‌شوند:

- **دستورات 20 و 14:** ابتدا ثابت R0 صفر شده و مقدار آن درون ثابت R2 کپی می‌شود. ثابت R2 در اینجا نقش جمله قبلی ($F_0 = 0$) را بازی می‌کند.
- **دستورات 05 و 0C:** یک واحد به R0 اضافه شده و در R1 کپی می‌شود تا جمله بعدی ($F_1 = 1$) شکل بگیرد.
- **دستورات 20 تا 1C:** برنامه با استفاده از جمع‌های متوالی و اضافه کردن عدد ۱، عدد ۱۰ را در R0 می‌سازد و سپس آن را در R3 کپی می‌کند. ثابت R3 از این پس به عنوان شمارنده حلقه عمل خواهد کرد.

۲. بدنه حلقه: محاسبه جمله جدید (آدرس‌های 0x0C تا 0x10)

این قسمت در هر تکرار، جمله جدید فیووناچی را محاسبه و ثابت‌ها را به‌روز می‌کند:

- **دستورات 01 و 02:** مقادیر R1 و R2 با هم جمع می‌شوند تا جمله جدید به دست آمده و در R0 قرار گیرد.
- **دستورات 32 و 0C:** مقادیر ثابت‌ها به‌روزرسانی می‌شوند؛ یعنی جمله فعلی به R2 منتقل شده و جمله جدید در R1 قرار می‌گیرد تا برای دور بعدی آماده باشند.

۳. کنترل حلقه و پرش (آدرس‌های 0x11 تا 0x17)

در این مرحله شمارنده چک می‌شود تا در صورت پایان یافتن ۱۰ مرحله، خروج از حلقه انجام گیرد:

- **دستورات 03، 25 و 1C:** مقدار شمارنده از R3 خوانده شده، یک واحد از آن کم می‌شود و مجدداً در R3 ذخیره می‌گردد.

- **دستور 99:** این یک دستور Jump if Zero است. اگر حاصل تفریق قبلی صفر شده باشد (فلگ Z یک باشد)، برنامه به آدرس 0x19 پرش کرده و از حلقه خارج می‌شود.

- **دستور EC:** اگر شرط بالا برقرار نشود، این دستور به صورت Unconditional Jump برنامه را مجدداً به ابتدای حلقه (آدرس 0x0C) بازمی‌گرداند.

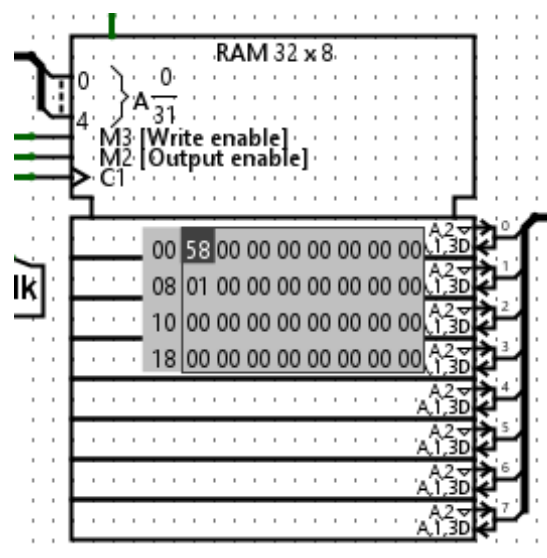
۴. محاسبه نهایی، ذخیره و توقف (آدرس‌های 0x19 تا 0x1D)

پس از ۱۰ بار چرخش در حلقه، ثابت R1 حاوی جمله دوازدهم فیبوناچی خواهد بود.

- **دستورات 01 و 25:** مقدار R1 به R0 منتقل شده و یک واحد از آن کم می‌شود (محاسبه $F_{12} - 1$).

- **دستور 60:** مقدار نهایی موجود در R0 (مجموع ۱۰ جمله اول) در آدرس صفر RAM با عملیات Store ذخیره می‌شود.

- **دستور FD:** برای جلوگیری از پیشروی PC در خانه‌های خالی ROM، این دستور یک پرش غیرشرطی به آدرس خودش (آدرس 0x1D) انجام می‌دهد و پردازنده را در یک حلقه بی‌نهایت متوقف می‌کند.



شکل ۱۷: خروجی نهایی ذخیره شده در RAM

۲.۴ تست دوم: جمع دو عدد ۶۴ بیتی

در دومین بخش، جمع دو عدد بزرگ ۶۴ بیتی مورد آزمایش قرار می‌گیرد. هدف این است که پردازنده دو عدد قرار گرفته در آدرس‌های ۰ و ۸ حافظه داده را خوانده و حاصل جمع آن‌ها را در آدرس ۱۶ به بعد ذخیره کند.

پردازش هر بایت از این دو عدد نیازمند یک بلوک ۵ دستوری است:

۱. بارگذاری بایت اول در ثبات R0

۲. کپی کردن مقدار R0 در R1 (استفاده از دستور 0C)

۳. بارگذاری بایت دوم در ثبات R0

۴. جمع کردن مقادیر دو ثبات (دستور 01)

۵. ذخیره حاصل از R0 در آدرس مقصد

کدهای هگزادسیمال مربوط به این عملیات را می‌توان بر اساس همین بلوک‌های پنج‌تایی تحلیل کرد. به عنوان مثال، بلوک اول (برای جمع بایت اول) به ترتیب شامل دستورات زیر است:

- دستور 40: بایت اول عدد اول را از آدرس 0x00 حافظه داده فراخوانی کرده و در ثبات R0 بارگذاری می‌کند.

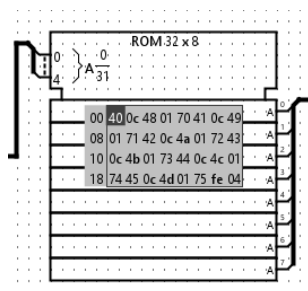
- دستور 0C: مقدار ثبات R0 را با عدد ثابت صفر جمع کرده و در ثبات R1 قرار می‌دهد (این کار در عمل محتوای R0 را درون R1 کپی می‌کند).

- دستور 48: بایت اول عدد دوم را از آدرس 0x08 حافظه خوانده و به ثبات R0 منتقل می‌کند.

- دستور 01: مقادیر موجود در ثبات‌های R0 و R1 با هم جمع شده و حاصل مجدداً در R0 ذخیره می‌گردد.

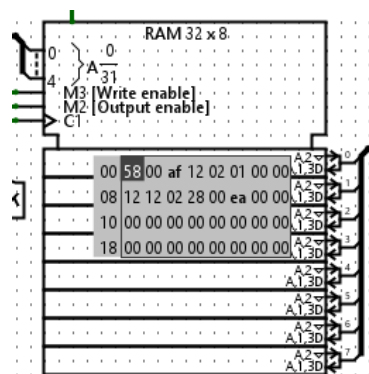
- دستور 70: نتیجه نهایی جمع از ثبات R0 به آدرس 0x10 منتقل و در حافظه RAM ذخیره می‌شود.

این الگوی پنج دستوری دقیقاً برای بایت‌های بعدی نیز تکرار می‌شود؛ برای مثال بلوک دوم به صورت 71 0C 49 01 نوشته شده است که آدرس‌ها در آن یک واحد افزایش یافته‌اند. در انتهای کدهای نوشته شده در حافظه برنامه، پردازنده با رسیدن به دستور FE (یک پرش غیرشرطی به آدرس خود دستور) به همراه دستور 04 به عنوان NOP در یک حلقه بی‌نهایت متوقف می‌گردد.



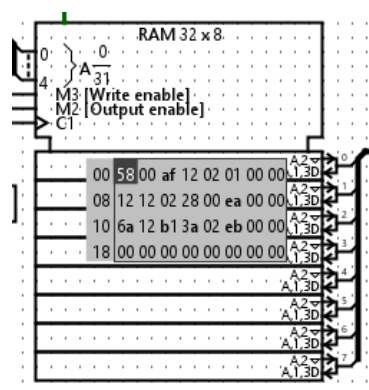
شکل ۱۸: کدهای هگزادسیمال برنامه در حافظه ROM

وضعیت RAM پیش از اجرای برنامه، نشان‌دهنده مقادیر اولیه دو عدد در آدرس‌های 0x00 و 0x08 است.



شکل ۱۹: وضعیت اولیه حافظه RAM پیش از اجرای محاسبات

پس از اعمال پالس‌های ساعت و اجرای کامل کدهای ROM، می‌توان نتایج را در بلوک سوم حافظه مشاهده کرد. پردازنده بایت‌ها را نظریه‌نظیر جمع کرده و در آدرس‌های مناسب قرار داده است. به عنوان نمونه در بایت اول، مجموع مقادیر 0x58 و 0x12 به درستی برابر با 0x6A محاسبه شده است.



شکل ۲۰: حاصل جمع در آدرس 0x10 پس از توقف پردازنده

۵ نتیجه‌گیری

در این آزمایش، ما توانستیم معماری کامپیوتر ساده خود را ارتقا داده و بر اساس معماری Harvard، قابلیت دسترسی به RAM و همچنین اجرای دستورات Jump (شرطی و غیرشرطی) را به آن اضافه کنیم. با این تغییرات، سیستم قادر شد مقادیر میانی را در حین اجرای برنامه ذخیره کرده و با استفاده از حلقه‌ها، جریان اجرای دستورات را به خوبی کنترل کند. مدار ما با تجزیه دستورات ۸ بیتی در زیرمدار Unpacker و بررسی فلگ‌های وضعیت در CU، عملیات درست را انتخاب می‌کند و با اعمال پالس سیگنال Clock، نتایج محاسبات را به درستی در حافظه یا رجیسترها ذخیره و به‌روزرسانی می‌کند.